

Lecture 01

MLIR 와 AI/NPU Compiler 큰 그림

MLIR for AI/NPU Compiler

기초부터 NPU lowering pipeline 까지 연결하는 첫 강의

**목표 : 모델 그래프를 NPU 명령 스트림으로 바꾸는 전체
경로를 한 장의 지도로 이해한다 .**

대상 : AI/NPU compiler 입문자

관점 : 256+ TOPS 급 Edge/Embedded NPU

산출물 : NPU compiler lowering pipeline 초안

20 강 전체 중 1 강은 “지도 만들기” 입니다.

오늘 강의의 역할

오늘은 MLIR 문법보다 compiler stack 전체 경로를 먼저 잡습니다.

2-5 강 : IR 구조, tool, dialect 기본기

6-17 강 : AI model IR 부터 tensor/memory/kernel optimization

18-20 강 : Custom NPU dialect, runtime ABI, capstone

핵심 질문

왜 하나의 IR 만으로는 AI/NPU compiler 가 어려운가 ?

StableHLO/TOSA/Linalg/MemRef/Vector/NPU Dialect 는 어디에 쓰이는가 ?

성능과 correctness 를 모두 만족하는 lowering pipeline 은 어떤 모양인가 ?

강의 후 스스로 설명할 수 있어야 하는 내용

1

AI/NPU compiler 의 end-to-end 역할을 설명한다 .

2

MLIR 가 multi-level IR + dialect ecosystem 인 이유를 이해한다 .

3

StableHLO/TOSA -> Linalg/Tensor -> MemRef/Vector/SCF -> NPU dialect 흐름을 그린다 .

4

Tiling, fusion, layout, memory space, command buffer 가 왜 중요한지 연결한다 .

5

MatMul 기반 NPU lowering pipeline 초안을 작성한다 .

NPU compiler 관점의 핵심 정의

MLIR 는 여러 abstraction level 을 하나의 compiler infrastructure 안에서 표현하고 , dialect 로 domain-specific IR 을 확장하며 , pass/rewrite/conversion 으로 점진적으로 lowering 하는 프레임워크입니다 .

High Level

모델 그래프 수준
Tensor/shape semantics
Framework portability

Mid Level

Loop/tile/memory 수준
Bufferization
Layout propagation

Low/Target Level

Vector/kernel 수준
Custom NPU ops
Runtime command ABI

Single IR, single pass 방식은 NPU 최적화에 부족합니다 .

문제

AI 모델은 graph-level semantics 와 tensor-level shape 정보가 중요합니다 .

NPU 는 SRAM, DMA, PE array, accumulator, quantization constraint 가 성능을 좌우합니다 .

High-level fusion 과 low-level memory scheduling 을 같은 pipeline 에서 연결해야 합니다 .

MLIR 는 dialect 별 abstraction 을 유지하면서 단계별 legality/correctness 를 검증할 수 있습니다 .

MLIR 식 해결

Frontend IR: 모델 의미 보존

Tensor IR: fusion/tile/vectorize 전 변환

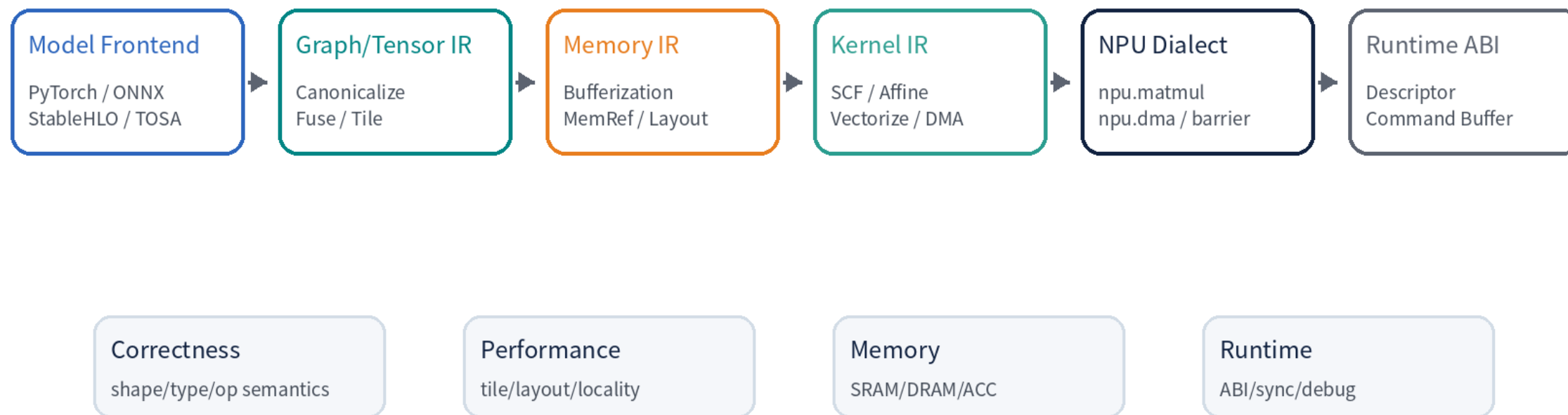
Memory IR: tensor 를 buffer 로 배치

Kernel IR: vector/loop/DMA scheduling

NPU IR: target-specific command generation

오늘 강의의 핵심 지도

MLIR-based AI/NPU Compiler Lowering Pipeline



2-3 강에서 상세 문법을 다룹니다 .

Operation

계산 / 제어 / 선언의 기본 단위

Value

SSA data edge

Type

shape/element/memory 표현

Attribute

layout/quant/tile metadata

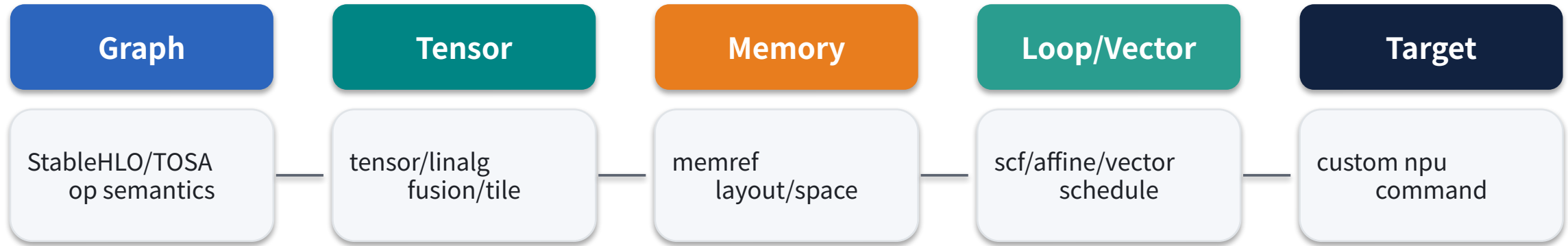
Region/Block

function/loop/graph body

Dialect

namespace + semantics package

정보 손실을 제어하면서 내려갑니다 .



NPU compiler 원칙 : 너무 빨리 low-level 로 내리면 fusion/shape 최적화를 잃고 , 너무 늦게 memory 를 보면 SRAM/DMA 제약을 놓칩니다 .

프레임워크와 NPU backend 사이의 공통 언어

StableHLO 관점

StableHLO: ML high-level operation semantics 를 안정적으로 표현

Framework portability 가 핵심

Backend 에서는 legal op set 으로 점진적으로 lowering

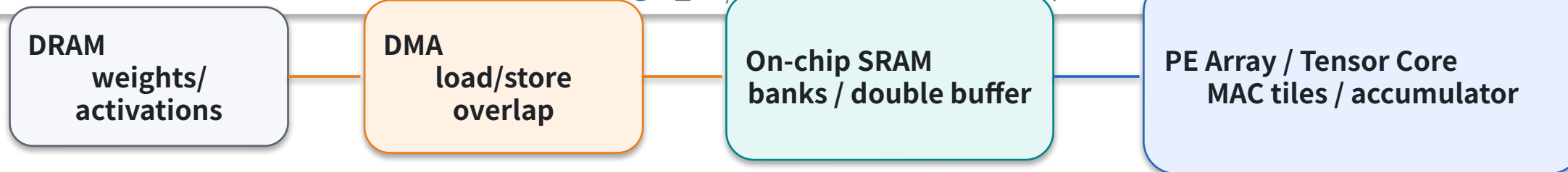
TOSA 관점

TOSA: DNN 에서 자주 쓰는 whole-tensor operation set

CPU/GPU/NPU target concern 을 고려한 tensor-level 표현

Edge NPU 에서는 op coverage 와 quantized semantics 가 중요

아키텍처 정보를 IR/Pass 에 반영해야 성능이 나옵니다.



Compiler-visible constraints

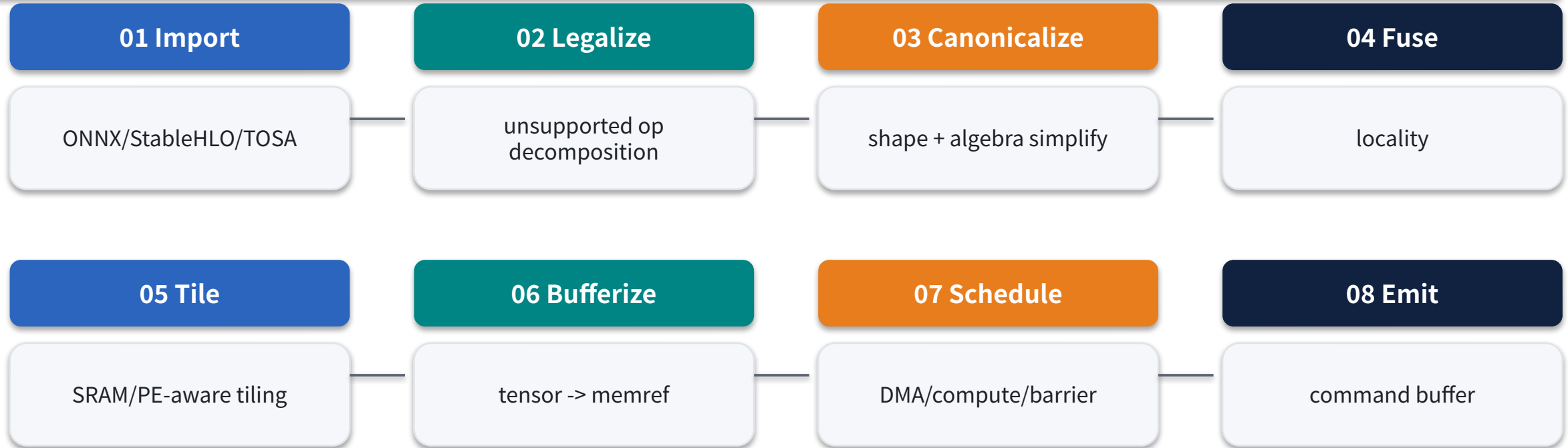
Tiling: M/N/K tile size 가 PE utilization 과 SRAM footprint 를 결정

Layout: blocked layout, weight packing, swizzle 가 DMA 효율에 영향

Scheduling: DMA load, compute, store, barrier 순서를 생성

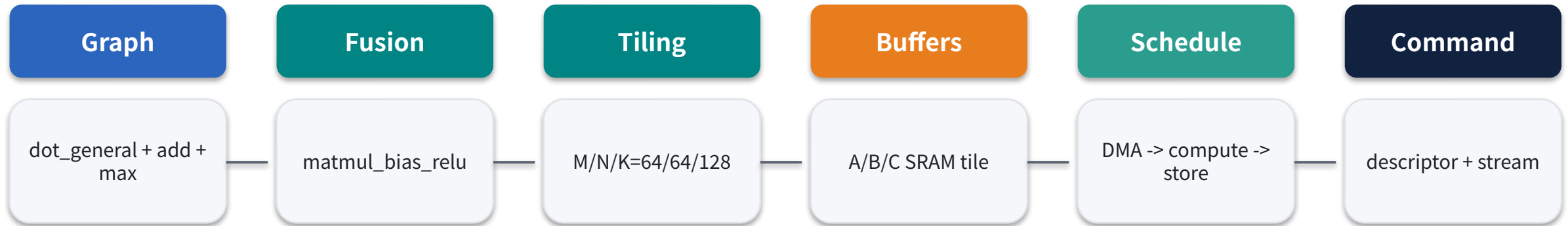
Quantization: INT8/INT4 입력 , accumulator, requant scale/zero-point 관리

1 강 실습에서 직접 작성할 결과물



중간 산출물은 항상 dump 가능해야 합니다 : IR after each pass, verifier result, shape/layout/memory report, performance estimate.

아주 단순한 op 도 여러 compiler 단계가 필요합니다 .



동시에 만족해야 하는 두 조건

Correctness: semantics, broadcasting, shape, activation clamp 유지

Performance: tile size, memory layout, prefetch distance, double buffering, accumulator spill 여부가 성능 좌우

다음 강의들의 navigation table

계층	대표 Dialect/IR	주요 정보	NPU compiler 포인트
Model	StableHLO/TOSA	op semantics, shape	frontend portability
Tensor	tensor/linalg	structured ops	fusion/tile candidate
Control	scf/affine	loop nest	tile loops, DMA order
Memory	bufferization/memref	buffer/layout/space	SRAM/DRAM mapping
Kernel	vector	vector.contract	tensorization
Target	custom npu	dma/matmul/barrier	command generation

실제 pass 이름은 project 별로 달라져도 구조는 유사합니다.

Pass pipeline shape:

model import

- > legalize model ops to tensor/linalg
- > canonicalize + CSE
- > fuse elementwise producers/consumers
- > tile for SRAM and PE array
- > bufferize tensor to memref
- > lower to loops/vector
- > insert DMA + barrier
- > emit NPU runtime ABI

1 강에서는 “이런 pipeline 이 필요하다” 를 이해합니다.

NPU-specific pass 는 legal op set, SRAM model, DMA model, quantization model 을 알아야 합니다.

좋은 compiler 는 빠르기만 해서는 안 됩니다 .

Correctness contract

- Shape/type consistency
- Broadcasting/quant semantics preservation
- Numerical tolerance and golden output comparison
- Deterministic runtime ABI
- Unsupported op fallback policy

Performance contract

- TOPS utilization
- DRAM bandwidth reduction
- SRAM bank conflict avoidance
- DMA/compute overlap
- Latency predictability for robotics/autonomous driving

NPU compiler debug 비용을 낮추는 설계

Input

shape/dtype/opset validation

After legalization

all ops legal or fallback-tagged

After fusion/tile

same semantics + valid tile coverage

After bufferization

alias/lifetime/memory space valid

After schedule

DMA order/barrier/dependency valid

After codegen

runtime descriptor matches binary

45-60 분 실습

실습 문제

입력 : MatMul + Bias + ReLU, batch=1, M=128, K=256, N=512

가정 : PE tile 64x64, K chunk 128, SRAM 2MB, INT8/FP16 지원

출력 : compiler pipeline 단계와 각 단계 산출물 정의

옵션 : DMA double-buffering 순서를 pseudo IR 로 표현

제출 산출물

표 1: pipeline 단계 / 입력 IR / 출력 IR / 목적

표 2: NPU target constraint / compiler decision

그림 1: DRAM-SRAM-PE dataflow

짧은 설명 : correctness check 3 개 + performance metric 3 개

NPU compiler 관점에서는 추상화 선택이 성능을 결정합니다 .

IR/Pass 실수

Graph IR 만 보고 backend 가 알아서 한다고 생각함
너무 빨리 low-level loop 로 내려서 fusion 정보를 잃음

Bufferization 전후 alias/lifetime 문제를 무시함
layout 을 마지막 codegen 단계의 문제로 미룸

NPU Mapping 실수

TOPS 만 보고 DRAM bandwidth 와 SRAM reuse 를 보지 않음

DMA/compute overlap 없는 schedule 생성
quantized accumulator/requant 범위 미검증
runtime ABI 와 compiler IR 의 연결 부족

오늘 반드시 가져갈 세 문장

1. MLIR 는 high-level model semantics 와 low-level NPU mapping 을 연결하는 multi-level infrastructure 입니다 .

2. NPU 성능은 fusion, tiling, layout, memory space, DMA schedule 을 lowering pipeline 에서 어떻게 결정하느냐에 달려 있습니다 .

3. 좋은 NPU compiler 는 pass 마다 IR dump, verifier, legality, numerical correctness, performance metric 을 남깁니다 .

다음 2 강 : Operation / Value / Type / Attribute 를 직접 읽고 간단한 MLIR 텍스트를 해부합니다 .

공식 문서 중심

1. MLIR Language Reference - <https://mlir.llvm.org/docs/LangRef/>

2. MLIR Understanding the IR Structure - <https://mlir.llvm.org/docs/Tutorials/UnderstandingTheIRStructure/>

3. MLIR Using mlir-opt - <https://mlir.llvm.org/docs/Tutorials/MlirOpt/>

4. MLIR Dialects - <https://mlir.llvm.org/docs/Dialects/>

5. MLIR Bufferization - <https://mlir.llvm.org/docs/Bufferization/>

6. StableHLO Specification - <https://openxla.org/stablehlo/spec>

7. TOSA - <https://www.mlplatform.org/tosa/>

8. MLIR TOSA Dialect - <https://mlir.llvm.org/docs/Dialects/TOSA/>